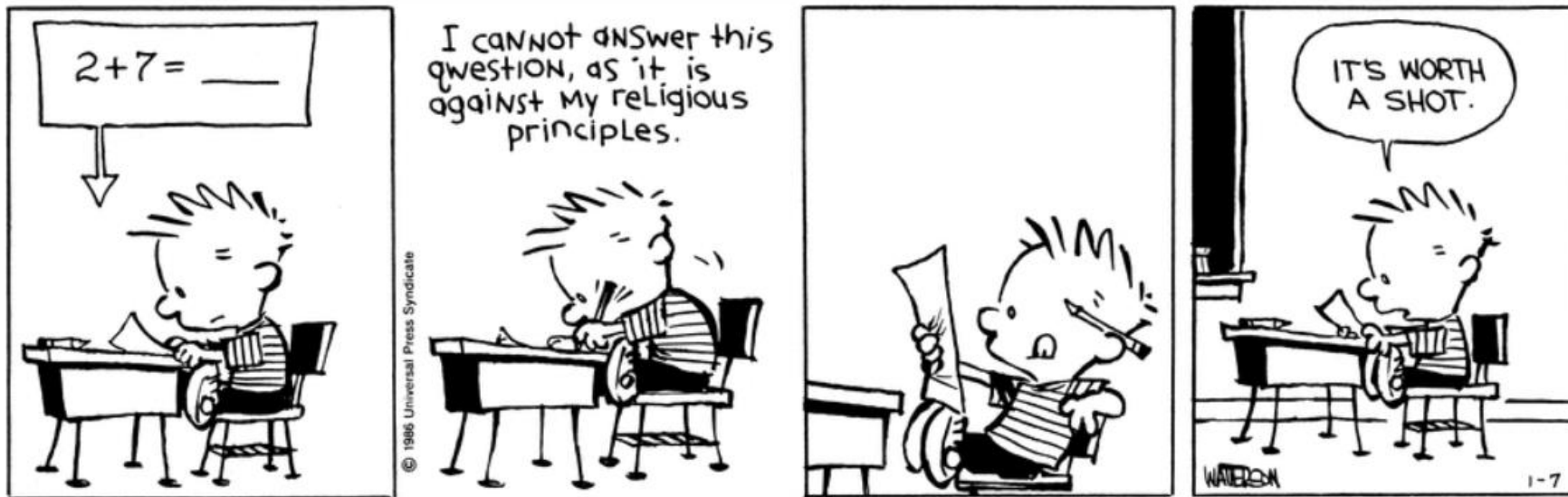


Lecture 3:

Variables + Statements



INFO 1100: Introduction to Programming

We've learned...

We've learned...

to **represent data!**

We've learned...

to **represent data!**

and **do math!**

We've learned...

to **represent data!**

and **do math!**

and **combine strings!**

but...

but...

what if we want to
store data?

Variables

Variables

What's a variable?

Variables

a name that refers to a value

Variables

a name that refers to a value

or

a named memory location
that contains a value

Variables

a name that refers to a value

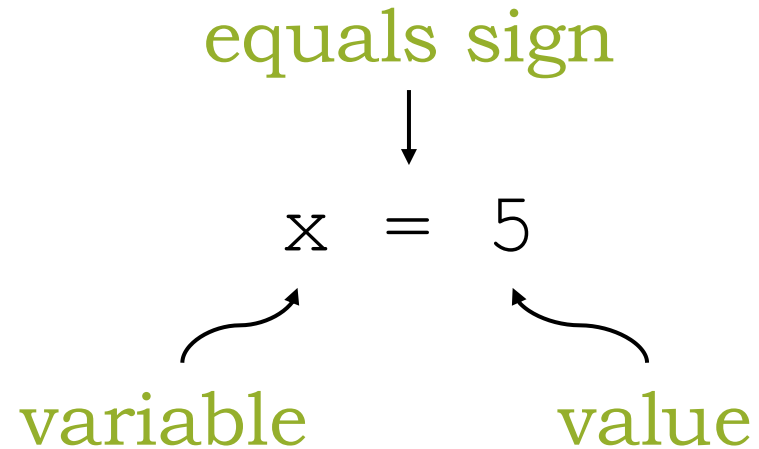
create a variable

$$x = 5$$

using an **assignment statement**

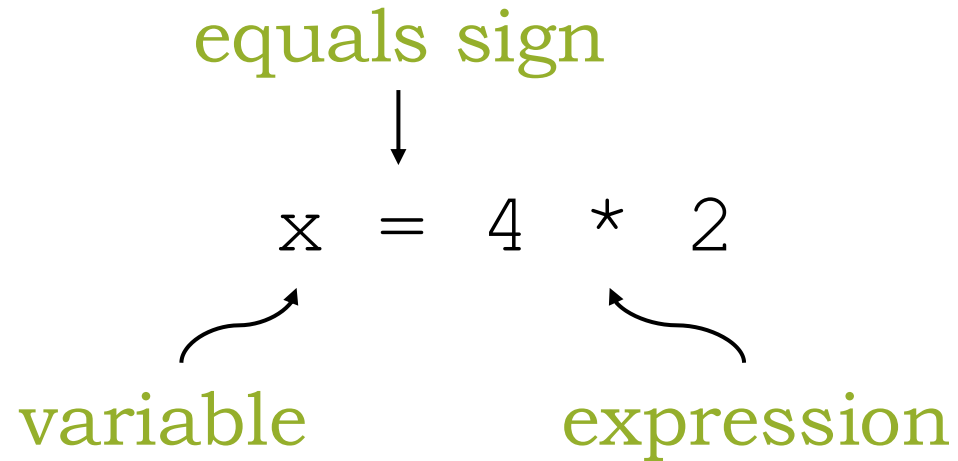
Variables

a name that refers to a value



Variables

a name that refers to a value



Variables

a name that refers to a value

assignment statement

$$x = 4 * 2$$

- takes the expression
 - evaluates it
- stores the value in the variable

Variables

a name that refers to a value

assignment statement

$$x = 4 * 2$$

- takes the expression
 - evaluates it
- stores the value in **memory**

Variables

a name that refers to a value

can appear in expressions!

$$(4 + x) / 3$$

Variables

a name that refers to a value

can be function arguments!

`len(g)`

Variable names

Variable names

the *name* of a variable is important!

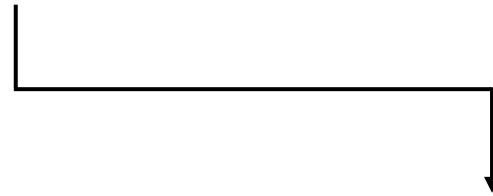
Variable names

the *name* of a variable is important!

Why do they matter?

Variable names

the **name** of a variable is important!



*helps you write clear +
understandable programs*

Variable names

the *name* of a variable is important!

Rules:

- contain letters + numbers
- only start with letters
 - only _ punctuation
- no keywords (e.g. `if`, `for`, `True`)

How else do we write
clear, comprehensible
code?

Comments

Comments

add *notes* to code

Comments

add **notes** to code

*also help you write clear +
understandable programs*

Comments

add *notes* to code

explain code behavior

Comments

add *notes* to code

start with #

Comments

add *notes* to code

start with #
everything after it is ignored

Comments

add *notes* to code

not using them almost
always leads to regret!

Statements vs. Expressions

Statements vs. Expressions

What's the difference?

Statements vs. Expressions

assignment *statement*

Statements vs. Expressions

statement

expression

Statements vs. Expressions

statement

- **does** something

expression

- **represents** something

Statements vs. Expressions

statement

- **does** something
- Python *executes* it

expression

- **represents** something
- Python *evaluates* it

Statements vs. Expressions

statement

- **does** something
- Python *executes* it
- results in an **action**

expression

- **represents** something
- Python *evaluates* it
- results in a **value**

Statements vs. Expressions

statement

- **does** something
- Python *executes* it
- results in an **action**
 - like a: *verb*

expression

- **represents** something
- Python *evaluates* it
- results in a **value**
 - like a: *noun*

Statements vs. Expressions

statement

- **does** something
- Python *executes* it
- results in an **action**
 - like a: *verb*

```
import math      x = 5
```

```
hi = "hello"
```

expression

- **represents** something
- Python *evaluates* it
- results in a **value**
 - like a: *noun*

Statements vs. Expressions

statement

- **does** something
- Python *executes* it
- results in an **action**
 - like a: *verb*

```
import math      x = 5
```

```
hi = "hello"
```

expression

- **represents** something
- Python *evaluates* it
- results in a **value**
 - like a: *noun*

```
"info" + "1100"      2**4
```

```
x == 5
```

import

import

what do these **statements** do?

`import`

allows you to access functionality from
modules

import

allows you to access functionality from
modules



collections of variables + functions

import

allows you to access functionality from
modules



collections of variables + functions



accessed via the **dot operator** (.)

import

allows you to access functionality from
modules

```
import math
```

import

allows you to access functionality from
modules

```
import math
```

```
math.pi = 3.14159...
```

```
math.factorial(3) = 6
```

```
math.ceil(2.3) = 3
```


import

allows you to access functionality from
modules

```
import math
```

```
math.pi = 3.14159...
```

expressions passed to
functions are
arguments

```
math.factorial(3) = 6
```

```
math.ceil(2.3) = 3
```

Debugging

Debugging

*it's very, very rare to write correct code
on your first try*

Debugging

*it's very, very rare to write correct code
on your first try*

therefore, you'll spend
a lot of time **debugging**

Debugging

printing relevant
values is one great
way to debug!

Debugging

printing relevant
values is one great
way to debug!

*are values what
you expect?*

*do you reach
certain lines of
code?*

Questions?

For tomorrow:

(06/25 @ 11:30am)

- Readings

Think Python –

Chapter 8.1-8.3, 8.5

- GRQs
- Lab 3

Sign up for
Friday quiz slot!